

Policy Gradients

A subset of policy optimization methods

Liz de Laittre - neuro study group - 3.30.26



State	Up	Down	Left	Right
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0
5	0	0	0	0
...	0	0	0	0
16	0	0	0	0

2

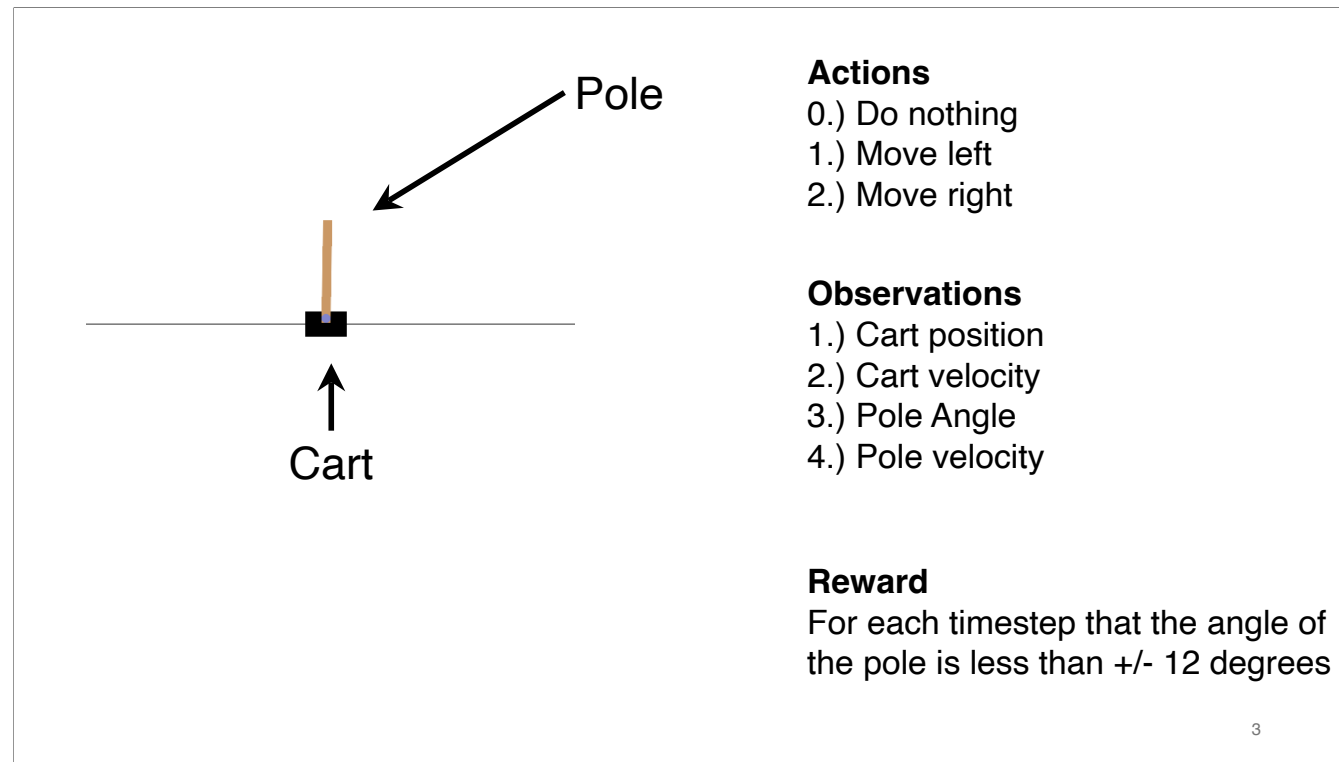
Tabular methods —> discrete states, discrete actions

Actions are both discrete AND unordered

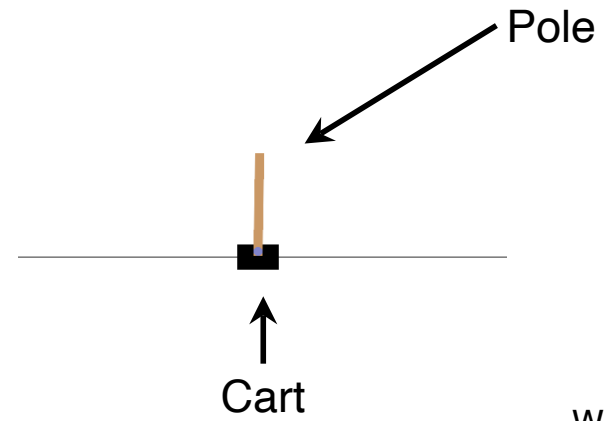
(tabular = table! Tabulate?)

The policy was just argmax over most valuable action (in a given state), which requires iterating over all states

Also our updates were to individual discrete states/actions, which meant that if there are similar actions, we cannot benefit from updating nearby ones (less sample efficient)



Continuous state space $\rightarrow$ no discrete states so we can't use action-value lookup table, instead need a function over continuous state space
Discrete action space



- Observations**
- 1.) Cart position
 - 2.) Cart velocity
 - 3.) Pole Angle
 - 4.) Pole velocity

What should our states be?

We no longer have discrete states...

But a continuous 4 dimensional space...

Discrete
state



Q
Table



Value of each
action

Continuous
state

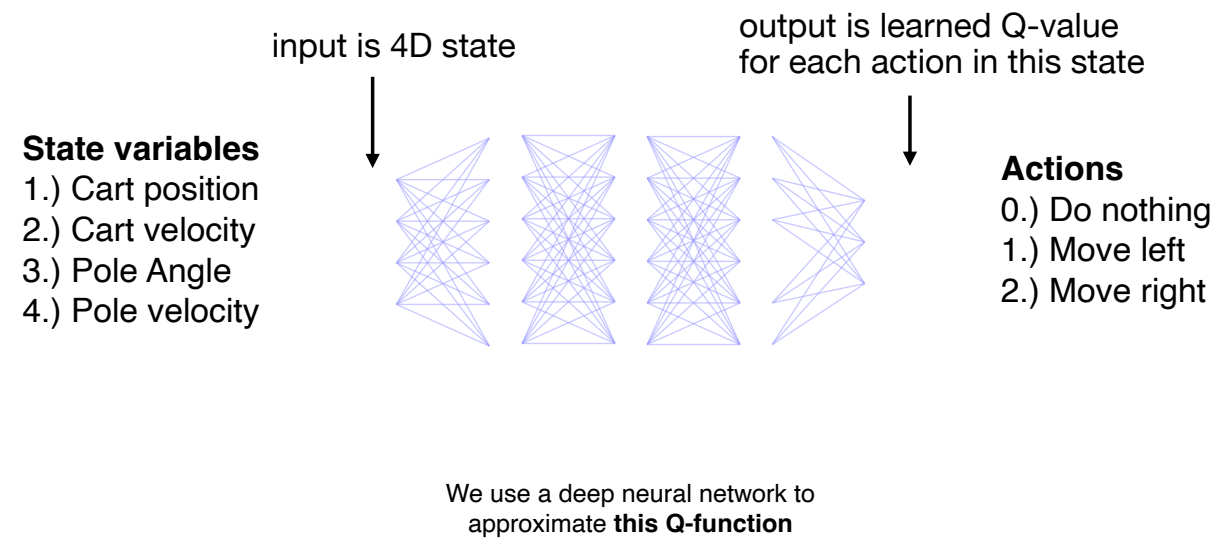


Some
function



Value of
each action

Continuous state space, discrete action space

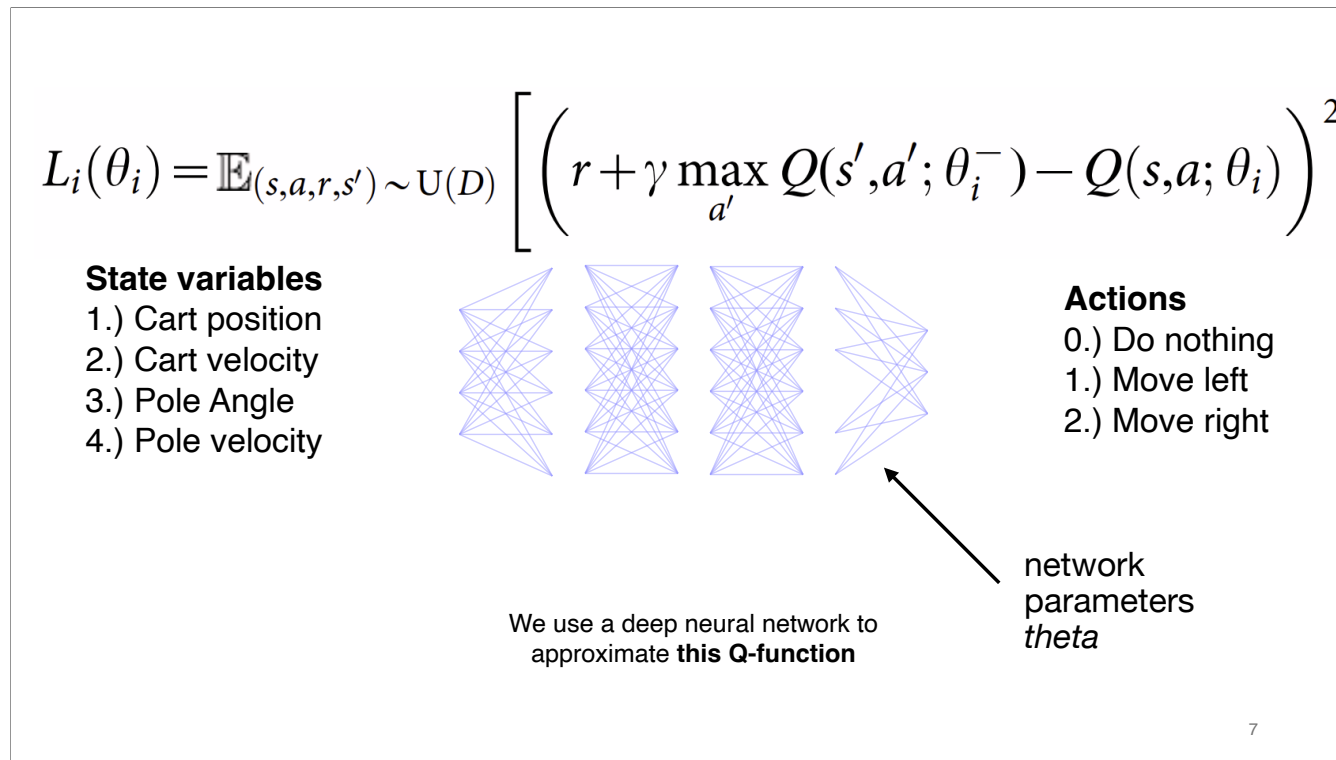


6

4D state landscape, mapping that space to a given discrete action, activations would be the value of that action at that point in state space. And then your policy would act on that value outcome, e.g. argmax or random

Goal is to learn the map of action-state to value, aka model of the environment, with a function approximator

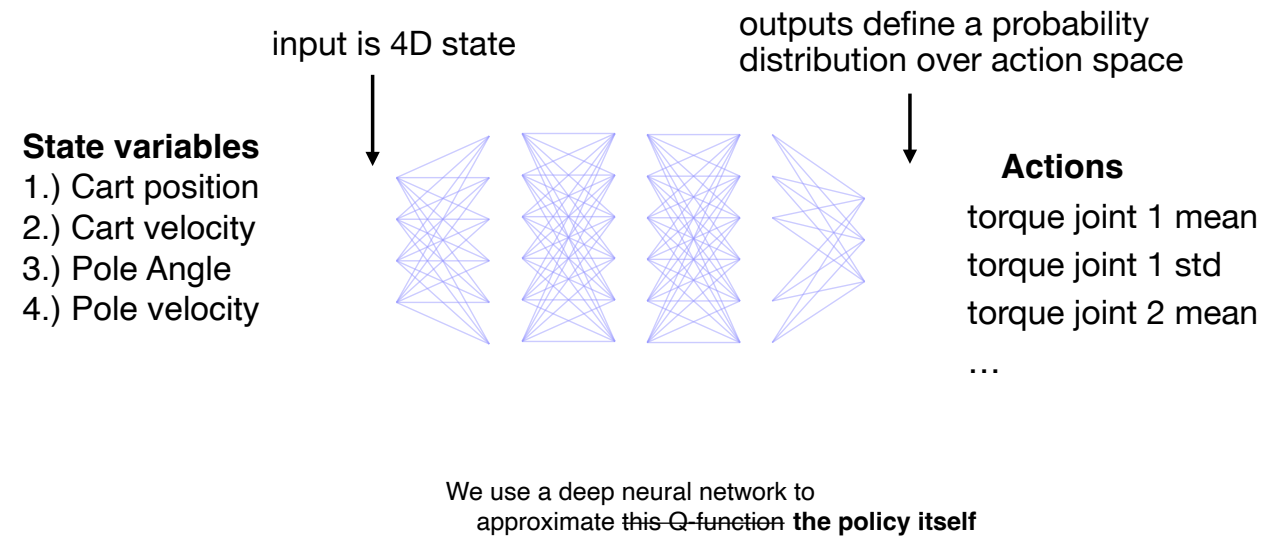
Computing this argmax over action



Loss function = updating the weights to approximate this Q value, in a way that maximizes total reward

Note that in computing Q learning, we are learning a deterministic state-action policy, since at every timepoint, the policy is to take the action with the highest value

Continuous state space, continuous action space



8

Policy gradient, still using a deep network, but change how we interpret the activations of output neurons, and our loss function, so how we update weights

Instead of updating weights to accurately represent the value of action-state combo (Q-value), you are updating weights to directly maximise total reward under that policy (or something else)

Q-value update is iterative, policy gradient is a gradient method

- Stochastic —> probability over actions, way you choose an action at each timestep is sampling from that probability distribution
- Discrete or continuous

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \nabla_{\theta} \mathbb{E}_{\tau \sim p_{\pi_{\theta}}(\tau)} [R_{\tau}] \quad \text{where } R_{\tau} := \sum_{t=0}^T \gamma^t r_t \\
&= \sum_{\tau} R_{\tau} \nabla_{\theta} p_{\pi_{\theta}}(\tau) \\
&= \sum_{\tau} R_{\tau} p_{\pi_{\theta}}(\tau) \nabla_{\theta} \log p_{\pi_{\theta}}(\tau) \\
&= \mathbb{E}_{\tau \sim p_{\pi_{\theta}}(\tau)} [R_{\tau} \nabla_{\theta} \log p_{\pi_{\theta}}(\tau)]
\end{aligned}$$

$J(\theta)$ = total reward for this stationary environment under policy π with parameters θ

Gradient with respect to the parameters θ

τ (trajectory through environment) drawn from the distribution of trajectories possible under policy π

$$\begin{aligned}
\nabla_{\theta} \log p_{\pi_{\theta}}(\tau) &= \nabla_{\theta} \left[\log p(s_0) + \sum_{t=0}^T \log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right] \\
&= \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).
\end{aligned}$$

Probability of a trajectory depends on environment as well as the policy

Simplify, since environment does not depend on parameters θ

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\pi_{\theta}}(\tau)} \left[R_{\tau} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \\
&\approx \frac{1}{N} \sum_{\tau \sim p_{\pi_{\theta}}(\tau)} \left(\sum_{t=0}^T \gamma^t r_t \right) \left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)
\end{aligned}$$

For stochastic policies, log-likelihood of action probabilities given the state

REINFORCE algorithm: change probability of actions-given-states in the direction that maximally increases total reward

Jensen, 2024 arXiv. An introduction to reinforcement learning for neuroscience

—> Gradient with respect to parameters of policy π (so in theory this could be small number, but usually it's a neural network)

Stochastic policy, continuous state and action space

Policy gradient methods maximize the expected total reward by repeatedly estimating the gradient $g := \nabla_{\theta} \mathbb{E} [\sum_{t=0}^{\infty} r_t]$. There are several different related expressions for the policy gradient, which have the form

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right], \quad (1)$$

where Ψ_t may be one of the following:

- | | |
|--|---|
| 1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory. | 4. $Q^{\pi}(s_t, a_t)$: state-action value function. |
| 2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t . | 5. $A^{\pi}(s_t, a_t)$: advantage function. |
| 3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula. | 6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual. |

The latter formulas use the definitions

$$V^{\pi}(s_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad Q^{\pi}(s_t, a_t) := \mathbb{E}_{s_{t+1:\infty}, a_{t+1:\infty}} \left[\sum_{l=0}^{\infty} r_{t+l} \right] \quad (2)$$

$$A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (\text{Advantage function}). \quad (3)$$

Applications in robotics and physics simulations

- <https://openai.com/index/learning-dexterity/>
- <https://gymnasium.farama.org/environments/mujoco/>

11

Commentary on why control is difficult

- movement command is not exactly what the body does (because of interaction with environment, also execution noise)
- Do the thing and see what you did (continuous update) —> sensory-motor delay ~100ms, which matters
- Forward models of how motor commands will be enacted, perform updates based on sensory predictions

Human and animal behavior compatible with policy-gradient RL

Bennet, Niv, and Langdon, 2021. COIBS. *Value-free reinforcement learning: policy optimization as a minimal model of operant behavior.* <https://doi.org/10.1016/j.cobeha.2021.04.020>

- Context-dependent preference learning (?)
- Continuous-action learning
- **Gideah and Pillow, 2025.** arXiv. *Inferring learning rules during de novo task learning.* <https://doi.org/10.1101/2025.09.29.679295>
- **Ashwood et al., 2020.** arXiv. *Inferring learning rules from animal decision-making* https://papers.nips.cc/paper_files/paper/2020/hash/234b941e88b755b7a72a1c1dd5022f30-Abstract.html

Question mark because their explanation seemed to imply that value-based RL only maps actions to value rather than allowing for state-dependent action-value mapping

Some properties of the policy gradient method

- Allows for stochastic policies —> balances exploration and exploitation
- Updates generalize to similar solutions (continuous space)
- On-policy only
- Model-free (no state transition probabilities)

Related topics

- Actor-critic methods —> policy gradient and value-based methods
- Deep Deterministic Policy Gradient (DDPG)
- Proximal Policy Optimization